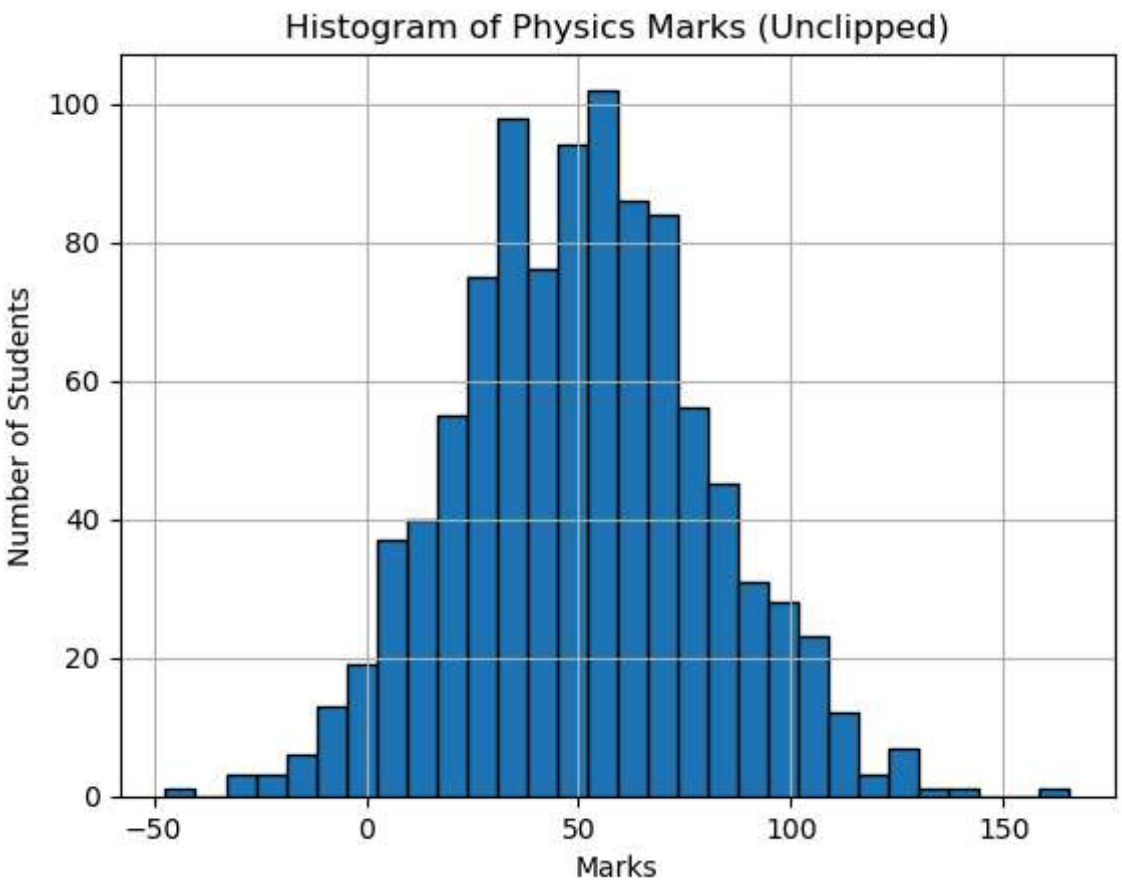
In [1]:
```
# Q2  1000 numbers from a normal distribution with a mean of 50 and a standard devi
```

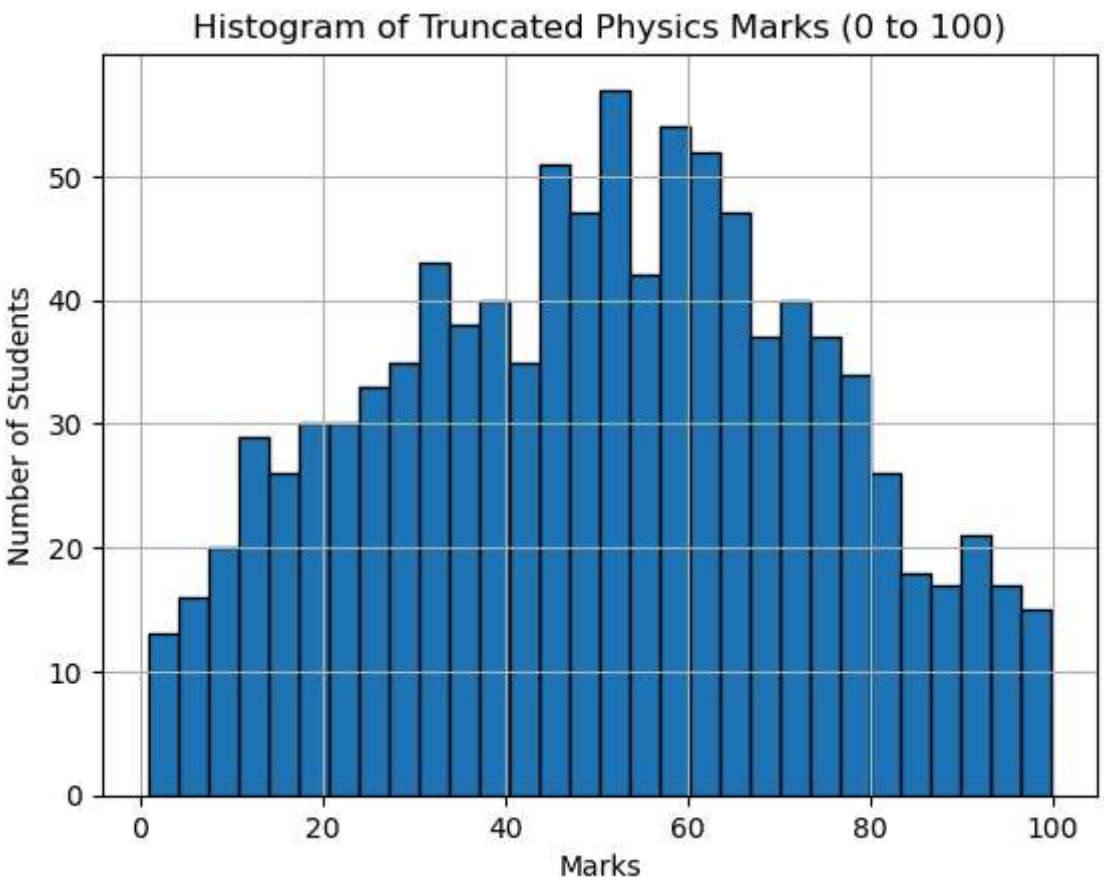
In [2]:
```
 import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import truncnorm

# Set random seed for reproducibility
np.random.seed(42)

# Normal distribution (mean = 50, std = 30)
raw_marks = np.random.normal(50, 30, 1000)

# Plot histogram
plt.hist(raw_marks, bins=30, edgecolor='black')
plt.title('Histogram of Physics Marks (Unclipped)')
plt.xlabel('Marks')
plt.ylabel('Number of Students')
plt.grid(True)
plt.show()
```



In [ ]:
```
# Modifed limit the distribution to the range of 0 to 100 using a set of truncated
```

In [3]:
```
# Parameters
mean = 50
std = 30
lower, upper = 0, 100
```

```
# Get truncated normal distribution
a, b = (lower - mean) / std, (upper - mean) / std
truncated_marks = truncnorm.rvs(a, b, loc=mean, scale=std, size=1000)

# Plot histogram
plt.hist(truncated_marks, bins=30, edgecolor='black')
plt.title('Histogram of Truncated Physics Marks (0 to 100)')
plt.xlabel('Marks')
plt.ylabel('Number of Students')
plt.grid(True)
plt.show()
```



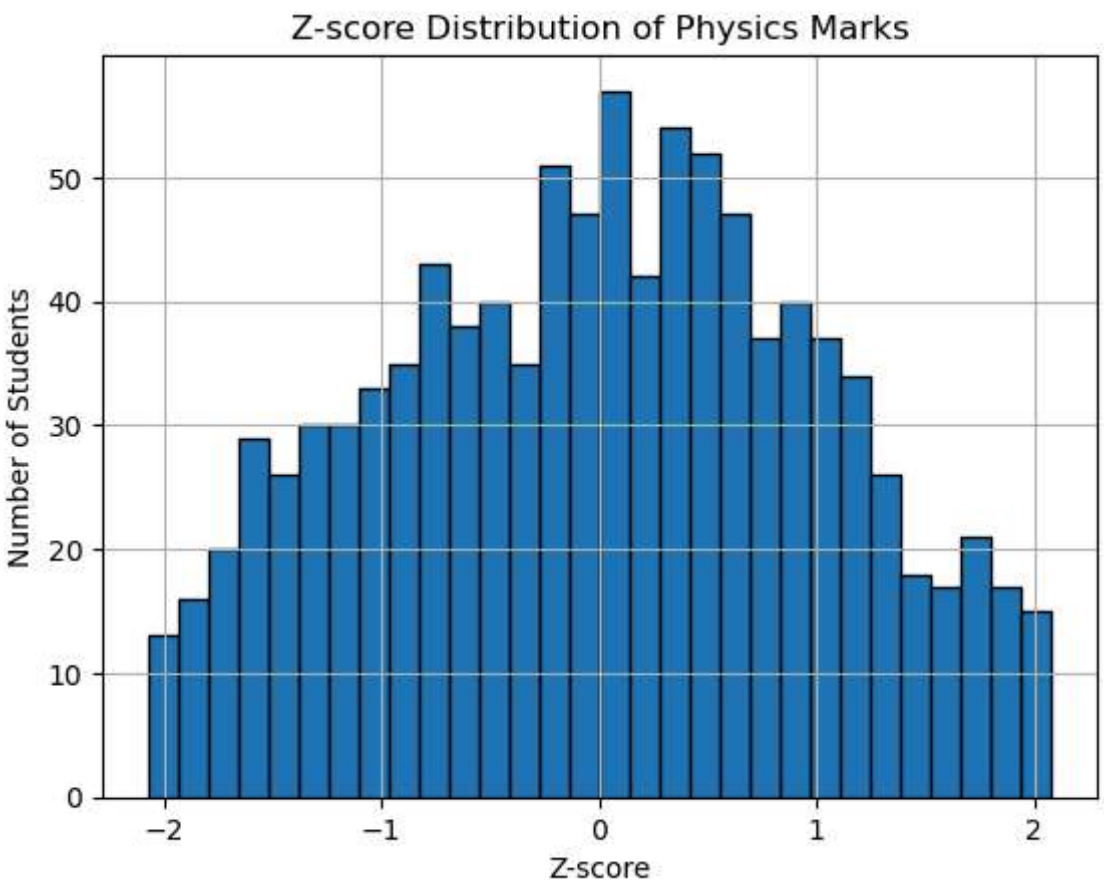
In [4]:
```
# calculated mean and the standard devition
```

In [5]:
```
# Print mean and standard deviation of raw marks
print("Raw Marks - Mean:", np.mean(raw_marks))
print("Raw Marks - Std Dev:", np.std(raw_marks))

# Print mean and standard deviation of truncated marks
print("Truncated Marks - Mean:", np.mean(truncated_marks))
print("Truncated Marks - Std Dev:", np.std(truncated_marks))
```

```
Raw Marks - Mean: 50.57996167466977
Raw Marks - Std Dev: 29.361786232420627
Truncated Marks - Mean: 50.243626621219484
Truncated Marks - Std Dev: 23.819926668033194
```

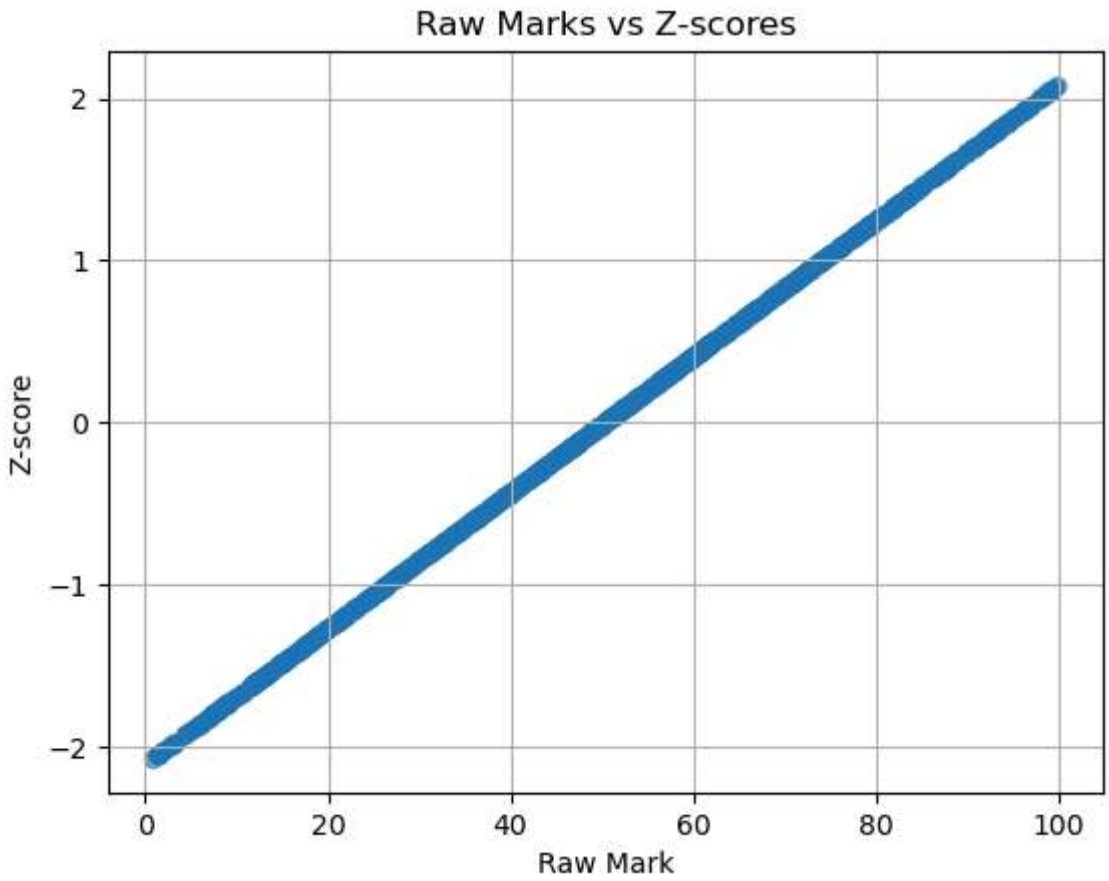
In [ ]:
```
#  Calculate Z-scores
```

In [7]:
```
mean_trunc = np.mean(truncated_marks)
std_trunc = np.std(truncated_marks)

z_scores = (truncated_marks - mean_trunc) / std_trunc
#print(z_scores)
# Plot Z-score distribution
plt.hist(z_scores, bins=30, edgecolor='black')
plt.title('Z-score Distribution of Physics Marks')
plt.xlabel('Z-score')
plt.ylabel('Number of Students')
plt.grid(True)
plt.show()
```



In [8]:
```
# Plot Raw Marks vs Z-scores:
```

In [9]:
```
plt.scatter(truncated_marks, z_scores, alpha=0.6)
plt.title('Raw Marks vs Z-scores')
plt.xlabel('Raw Mark')
plt.ylabel('Z-score')
plt.grid(True)
plt.show()
```



In [ ]: